

C++ 추가 학습

장비·머신비전 소프트웨어 이직을 위한 구현과 검증

현재 자료 다음에, 무엇을 직접 해볼까

현재 자료의 원리를 작은 C++ 검사 서비스에 적용하고, 종료·과부하·장애를 재현 가능한 증거로 설명하는 추가 학습서입니다. 카메라 SDK 없이 실행하는 시뮬레이터부터 시작합니다. 주 6~8시간은 이 폴더에 집중할 때의 제안이며, 6주를 고정 마감으로 보지는 않습니다.

기준 폴더	CS / CPP
검토 자료	Markdown 9개 · 기존 PDF 3개
기준 직무	C++ 기반 장비·머신비전 소프트웨어 개발자
권장 활용	주 6-8시간을 이 폴더에 배정하는 6주 집중 과정

읽기 → 구현·실험 → 결과 설명

자료 보유와 학습 완료는 별개입니다. 각 장의 통과 기준으로 시작 위치를 정하세요. 세 폴더의 계획은 개별 집중 기준이며, 병행할 때는 총 가용 시간에 맞춰 기간을 늘립니다.

01 / 출발점

읽은 원리를 증명 가능한 구현으로

현재 폴더는 기초 설명보다 제품 코드에 가까운 검증 실습을 추가할 시점이다.

00~04에는 메모리·수명·실행 모델·동기화, 05~08에는 SDK 버퍼·의존성 분리·비동기 UI·다중 카메라 예제가 있다. 기존 PDF 3권도 이 내용을 묶는다. 따라서 RAII나 mutex 정의를 다시 쓰는 대신, 이미 제시된 설계를 다른 사람이 실행하고 반례로 검사할 수 있게 만드는 데 시간을 쓴다.

기존 근거	추가로 만들 증거
03: bounded queue	수락·거부·완료 취소가 빠짐없이 맞는 프레임 정산
05: 종료 상태 머신	콜백 완료 확인, 종료 실패 상태, 재접속 세션 격리
06: fake와 테스트	CMake/CTest 실행 명령 및 같은 순서로 재현되는 실패
08: 멈춤 진단	실제 trace·스택·측정 조건을 묶은 원인 보고서

자료에서 CMake/CTest 구성과 위 산출물의 완성본은 확인되지 않았다. 이는 현재 실력을 단정한 평가가 아니다. 이미 구현해 본 항목은 통과 기준을 확인한 뒤 건너뛰면 된다. 우선순위는 재현 빌드 → 입력·오류 계약 → 종료·정산 테스트 → 원인 진단 → 성능 비교 순서다.

이번 폴더의 실습 대상

FakeCamera가 정해진 이미지와 식별자를 만들고, bounded queue와 worker가 간단한 checksum 검사를 수행하며, 결과를 메모리 sink에 보관하는 콘솔 프로그램을 만든다. UI·실카메라는 나중에 adapter로 연결한다. checksum은 시스템 검증용 대역이며 검사 정확도나 CV 알고리즘 성능의 증거로 사용하지 않는다.

첫 통과 기준

다른 경로에 복제해 빌드·테스트할 수 있고, 과부하와 종료 경쟁을 의도적으로 발생시켜 결과를 설명할 수 있다. 실제 지원 직무의 OS·SDK·C++ 버전이 정해지면 adapter와 빌드 조건만 조정한다.

02 / 미니 레슨 1

빌드도 재현 가능한 설계다

IDE 설정에 숨어 있던 의존성과 테스트 실행 방법을 저장소의 계약으로 만든다.

src/ledger.cpp, include/ledger.hpp, tests/ledger_test.cpp로 정산 규칙을 분리한다. CMake의 PUBLIC은 사용자가 필요로 하는 조건을 전파하고 PRIVATE은 해당 target에 적용한다. 공개 헤더가 C++20 타입을 쓰면 그 요구를 PUBLIC으로 표현한다. 전역 include 경로를 누적하기보다 target별 의존성을 선언한다.[1]

```
cmake_minimum_required(VERSION 3.23)
project(inspection LANGUAGES CXX)
add_library(core src/ledger.cpp)
target_include_directories(core PUBLIC include)
target_compile_features(core PUBLIC cxx_std_20)
set_target_properties(core PROPERTIES CXX_EXTENSIONS OFF)
include(CTest)
if(BUILD_TESTING)
  add_executable(core_tests tests/ledger_test.cpp)
  target_link_libraries(core_tests PRIVATE core)
  add_test(NAME ledger COMMAND core_tests)
  set_tests_properties(ledger PROPERTIES TIMEOUT 10)
endif()

# Configure; then build and test the same configuration.
cmake -S . -B build -DBUILD_TESTING=ON
cmake --build build --config Debug
ctest --test-dir build -C Debug --output-on-failure
```

위 CMake 파일과 아래 셸 명령은 서로 다른 위치에 쓴다. 테스트 실행 파일은 성공 시 0, 불변식 위반 시 0이 아닌 값을 반환하게 구현한다. Release에서 제거될 수 있는 assert만으로 채점하지 않는다. 10초는 학습용 멈춤 방지 상한이며 처리 속도 합격 기준이 아니다.

공통 configure/build/test 설정은 CMakePresets.json에, 개인 SDK 경로는 버전 관리하지 않는 CMakeUserPresets.json에 둔다.[2] 다음 단계로 core의 스레드 사용을 추가할 때 Threads target을 연결하고, 실카메라 adapter는 별도 target으로 분리한다.

실습과 채점

원본 build 폴더를 사용하지 않는 새 경로에서 동일 명령을 실행한다. 정산식 하나를 의도적으로 틀리게 만들면 CTest가 실패해야 한다. 성공·실패 로그와 compiler/CMake 버전을 남기면 통과다. 단순히 실행 파일이 커지는 것은 테스트 통과가 아니다.

03 / 미니 레슨 2

유효한 Frame만 경계를 통과시킨다

소유권을 갖는 것과 올바른 크기·의미를 갖는 것은 별도의 조건이다.

08번 예제의 Frame에 width, height, stride, pixel_format, session_id, frame_id, recipe_version을 추가한다. SDK adapter는 빌린 바이트를 검증한 뒤 owned Frame을 생성한다. 검사가 거부되면 큐·recipe·기존 결과를 변경하지 않는다는 계약을 정한다. 공개 필드를 아무렇게나 바꾸기보다 생성 함수에 불변식을 모은다.

```
#include <cstdint>
#include <variant>
enum class SubmitError {
    InvalidShape, UnsupportedFormat, Full, Stopping
};
struct Accepted { std::uint64_t ticket; };
using SubmitResult = std::variant<Accepted, SubmitError>;

// Gray8 only; positive width, height, stride.
// min_bytes = (height - 1) * stride + width
// Check overflow BEFORE multiplication and addition.
// Require stride >= width, buffer.size() >= min_bytes.
// Accepted transfers ownership to the pipeline.
```

위 크기식은 마지막 행 뒤의 padding까지 요구하지 않는 Gray8 view의 최소 읽기 범위다. 앱이 height×stride 전체를 복사한다면 더 큰 저장 공간 계약을 사용해야 한다. packed 10/12-bit, 음수 stride, 다중 plane은 이 공식을 그대로 쓰지 말고 지원하지 않는 입력으로 분리한다. SDK가 보고한 길이와 주소의 유효 기간도 별도로 확인한다.

오류의 의미를 고정하는 실습

- 정상 4×3 Gray8, stride 8, 20바이트 입력은 마지막 픽셀까지 읽을 수 있다. 19바이트, stride 3, height 0은 거부되어야 한다.
- size_t 상한 근처 입력으로 곱셈·덧셈 overflow를 검사한다. 큰 메모리를 실제 할당하지 않고 검증 함수를 직접 테스트한다.
- Full은 제출 실패, Cancelled는 수락 뒤 작업의 종료 이유로 분리한다. 중복 terminal 결과가 생기지 않게 ticket마다 한 번만 종결한다.

설계 판단과 통과 기준

C++20에서는 variant 등으로 값/오류를 표현한다. C++23 expected로 바꾸려면 대상 컴파일러와 표준 라이브러리 지원을 먼저 확인한다. 오류 종류·재시도 여부·상태 변경 여부를 한 장에 설명하고, 거부 입력이 큐 길이와 기존 결과를 바꾸지 않으면 통과다.

04 / 미니 레슨 3

종료 성공과 종료 실패를 구분한다

기존 종료 순서에 완료 조건과 재접속 규칙을 붙인다.

05번은 종료 단계, 08번은 callback barrier의 필요성을 이미 설명한다. 이번에는 각 단계가 끝났다는 증거를 API 결과로 만든다. 단순한 stop flag를 Stopped와 같은 의미로 사용하지 않는다. 상태 변경은 한 제어 경로로 직렬화하고, 외부 SDK 호출이나 join은 상태 mutex 밖에서 수행한다.

상태 전이	완료 조건과 다음 행동
Running → Stopping	새 수락 차단과 stop 요청을 기록한다. 중복 stop은 같은 진행 중 결과를 관찰한다.
Stopping → Stopped	SDK의 새 callback 금지 보장 + 진입한 callback 0 + worker join + retained buffer 0을 확인한다.
Stopping → Faulted	deadline 안에 완료를 확인하지 못했다. 남은 작업·handle·buffer를 살아 있게 유지하고 원인을 기록한다.
Faulted → Starting	기존 세션의 callback과 자원 사용이 종료된 뒤 명시적으로 새 세션을 연다. 자동 재시도 횟수·간격은 설정한다.

진입 카운터가 순간적으로 0인 것만으로 callback 종료를 증명할 수 없다. SDK가 앞으로 callback을 호출하지 않는다는 unregister/stop 계약과 함께 확인해야 한다. gate 검사와 진입 예약은 같은 동기화 규칙으로 묶고 callback context는 SDK가 접근할 가능성이 사라질 때까지 유지한다.

Windows CanceledEx도 취소를 요청할 뿐 완료를 기다리지 않는다.[3] 반환 직후 버퍼나 OVERLAPPED를 재사용하지 않는다. 취소 불가능한 외부 호출이 있다면 이 구조만으로 유한 시간 종료를 보장할 수 없다. deadline 초과를 성공으로 바꾸거나 살아 있는 객체를 해제해서 해결하지 않는다.

실습과 기대 관찰

FakeCamera가 종료 직전에 callback을 보류하게 한다. stop 중 frame이 거부되고 callback 해제 뒤에만 Stopped가 되는지 확인한다. 재접속은 session_id를 증가시키며 이전 세션 결과를 반영하지 않아야 한다. 촬영 trigger처럼 중복 실행에 의미가 있는 명령은 실패 응답만 보고 자동 재전송하지 않는다.

05 / 미니 레슨 4

사라진 프레임을 숫자로 찾는다

bounded queue의 용량 제한에 프레임 생애의 정산 불변식을 더한다.

관측 범위는 앱에 들어온 제출 시도다. SDK 이전의 전송 손실은 별도 지표이며 아래 식으로 측정할 수 없다. 수락한 ticket은 정확히 한 상태에 속한다. 전체 카운터는 같은 시점의 snapshot으로 읽거나 실행 종료 뒤 대조한다. 서로 다른 atomic 값을 제각각 읽은 중간 합계로 유실을 단정하지 않는다.

```
received = accepted + rejected
accepted = queued + running + succeeded
           + failed + cancelled + expired
rejected = full + invalid + stopping + duplicate

// Terminal states are mutually exclusive.
// Preview eviction after acceptance => expired.
// A hidden/stale UI result is a separate UI metric.
```

검사 완료와 UI 표시를 분리하면, 화면에 안 보였다는 이유로 계산 성공을 실패로 다시 세는 실수를 피할 수 있다. 완료의 의미는 이번 실습에서 계산 결과가 result sink에 기록된 시점으로 고정한다. 저장에 필수인 제품이라면 그 완료 조건과 저장 실패 상태를 다시 설계해야 한다.

과부하를 계산하고 정책을 선택하기

일정한 입력 120 frame/s, 처리 100 frame/s라는 단순 모델에서는 초당 20장이 쌓인다. 빈 큐의 용량 8장은 약 0.4초의 여유일 뿐, 지속 과부하를 해결하지 못한다. 필수 검사는 입력 제어 또는 명시적 거부를, preview는 교체·폐기를 선택한다. SDK callback을 막아도 장비까지 역압력이 전달되는지는 SDK 계약으로 확인한다.

2048×2048 Gray8 한 장은 4 MiB다. queued 8장 + running 2장 + 수신 복사 중 1장이면 앱 입력 버퍼만 44 MiB가 필요하다. SDK pool, 출력 이미지, UI 대기분은 추가된다. 큐 8개라는 설정만으로 프로세스 전체 메모리 상한이 정해지는 것은 아니다.

실습과 채점

수신률과 처리 지연을 바꾸며 queue high-water mark, oldest age, 거부 사유를 기록한다. 종료 뒤 queued=running=0이고 두 정산식이 맞아야 한다. 카메라별 식도 확인해 특정 카메라만 계속 밀리는 현상을 전체 평균에 숨기지 않는다.

06 / 미니 레슨 5

종료 경쟁을 원하는 순서로 재현한다

06번의 fake 설계와 08번의 stress 목록을 한 개의 실패 가능한 시나리오로 완성한다.

운 좋게 1,000번 성공하는 테스트보다 특정 순서에서 반드시 틀린 구현을 잡는 테스트를 먼저 만든다. worker가 처리 시작 직후 도착 사실을 알리는 latch와 진행을 허용하는 gate를 테스트 경계에 둔다. 제어 thread가 gate를 해제하기 전 worker가 자원이나 상태를 먼저 정리하지 못하게 한다.

의도한 실행 순서	확인할 상태
용량 1, worker 1로 frame A 제출	A가 running이 된 뒤 gate에서 대기
B 제출, 이어서 C 제출	B는 queued, C는 Full로 거부
cancel 방식 stop 요청	새 수락 차단, B는 cancelled
gate 해제, A가 취소 확인	A도 cancelled, worker 종료
stop 완료 뒤 snapshot	R=3, A=2, J=1, cancelled=2

이 시나리오에서는 checksum 계산 전 checkpoint가 취소를 우선 확인하도록 계약한다. 그래서 succeeded=0, queued=running=0을 정확히 기대할 수 있다. gate 대기는 테스트용이며 제품의 취소 프로토콜을 대신하지 않는다. 실패 경로에서도 gate를 해제해 테스트 정리 자체가 교착하지 않게 한다.

두 종류의 추가 반례

- publish 직전 보류 → recipe 교체 → 재개: 계산 완료 수와 별도로 stale UI commit 수를 검증한다. 새로운 화면을 이전 결과가 덮으면 실패다.
- 특정 ticket의 inspector에 예외 주입: 해당 작업만 failed로 종결하고, 정책이 계속 처리라면 다음 작업은 성공해야 한다. SDK 경계 밖으로 예외가 새지 않는지도 확인한다.

결정적 시나리오가 통과한 뒤 worker 수·큐 용량·입력 순서를 바꾼다. 난수 seed는 입력을 재현하지만 OS 스케줄까지 같게 만들지는 않는다. 실패한 event 순서와 주입 지점을 함께 저장해야 원인을 다시 추적할 수 있다.

통과 기준

임의 sleep 없이 상태 조건으로 진행한다. 잘못된 구현에서는 테스트가 실패하고, 수정 뒤에는 같은 입력과 순서에서 통과해야 한다. 외부 timeout은 영구 멈춤을 끊는 안전장치이며 성공 판정은 불변식으로 한다.

07 / 미니 레슨 6

장애 하나에 재현 묶음 하나를 남긴다

도구 이름을 아는 단계에서, 증상과 원인을 연결하는 보고서를 만드는 단계로 간다.

증상이 생기면 commit ID, compiler/SDK 버전, build 종류, 설정 파일, 입력 식별자, 발생 순서를 묶는다. 각 event는 monotonic time, thread, session, camera, ticket, stage를 가진다. 서로 다른 장치의 시계를 직접 빼지 말고 앱 안의 같은 시계로 구간 시간을 재거나 동기화 조건을 명시한다.

관찰	다음에 확보할 증거
크래시·잘못된 메모리	ASan 보고서의 잘못된 접근과 할당·해제 스택을 연결한다.
낮은 CPU·진행 정지	모든 thread stack과 대기 자원 관계를 수집한다. CPU만으로 교착을 단정하지 않는다.
높은 CPU·큐 누적	CPU hot path, 처리량, oldest age로 계산 병목과 입력 과부하를 구분한다.
결과만 가끔 틀림	ticket별 recipe·세션·buffer 수명과 결과 전달 순서를 비교한다.

Windows/MSVC는 별도 ASan 구성에서 메모리 오류를 검사한다. /fsanitize=address 사용 조건을 확인하고, instrument된 빌드의 지연을 제품 성능값으로 기록하지 않는다.[4] ThreadSanitizer 공식 지원 목록에는 Windows가 없으므로, 플랫폼에 독립적인 core를 Linux/Clang 환경에서 별도로 검사하는 방법을 선택할 수 있다.[5] 그 결과는 Windows SDK 내부까지 검사했다는 뜻이 아니다.

실습: 문제 보고서 한 장

수명 오류 또는 잘못된 세션 결과 반영 중 하나를 격리된 테스트에 의도적으로 넣는다. 보고서에 재현 명령, 기대값, 실제값, 최초로 깨진 불변식, 원인 코드, 수정, 회귀 테스트를 쓴다. 스택과 dump는 같은 빌드의 실행 파일·심벌과 함께 보관해야 나중에 함수 위치를 해석할 수 있다.

채점 기준

누군가 보고서만 읽고 같은 실패를 다시 만들 수 있어야 한다. sanitizer 무보고는 모든 오류가 없다는 증거가 아니다. 메모리 오류·data race·논리적 순서 위반·SDK 계약 위반 중 무엇을 확인했고 무엇이 남았는지 구분해 적는다.

08 / 미니 레슨 7

빠르다는 주장에 비교 조건을 붙인다

기존 성능 지표를 실제 입력·구간·분모가 있는 실험으로 바꾼다.

처음에는 복사 기반 버퍼와 mutex 큐를 기준으로 삼는다. profiler가 병목을 가리키기 전에는 lock-free, custom allocator, coroutine을 동시에 넣지 않는다. 변경 이유와 비용을 분리하기 위해 한 번에 하나만 바꾸고, 동일 입력과 정산 결과가 유지되는지 먼저 확인한다.

```
t0 = callback entry      t1 = accepted
t2 = worker start        t3 = result recorded
t4 = UI commit

queue_wait = t2 - t1
service    = t3 - t2
end_to_end = t3 - t0
ui_delay   = t4 - t3
// Collect with one monotonic application clock.
```

입력률, 이미지 크기, worker 수, 큐 용량, warm-up, 실행 시간, Release 설정, 로그 수준을 기록한다. completed frame 지연만 개선하고 거부율이 늘면 전체 시스템이 좋아졌다고 말할 수 없다. 처리량·거부율·p50/p95/p99·최대 대기 나이·메모리 high-water mark를 함께 본다. 표본 수가 작은 p99는 특히 불안정하므로 표본 수와 계산 방식을 남긴다.

실험 A: 할당을 재사용해 보기

프레임마다 vector를 만드는 기준선과 고정 수 버퍼를 빌려 반환하는 pool을 비교한다. checkout/release 수, 대여 중 수, pool 소진 횟수를 측정한다. pool이 비면 입력 거부·대기 중 무엇을 하는지 명시한다. 반환 전 buffer 재사용이 없고, 중단 뒤 outstanding=0이어야 성능 결과를 채택한다.

실험 B: worker 수를 늘려 보기

worker 1·2·4개를 같은 총입력으로 비교한다. 처리량이 멈추고 지연만 커지면 메모리 대역폭·lock-SDK 또는 다른 라이브러리 내부 thread 경쟁을 확인한다. 카메라 두 개의 합계뿐 아니라 각 카메라의 지연도 기록한다. checksum 결과만으로 실제 CV 알고리즘의 확장성을 일반화하지 않는다.

통과 기준

최소 세 번의 독립 실행을 시작점으로 원시 CSV와 분산을 남긴다. 이는 제안한 학습 기준이다. 개선이 없으면 그대로 보고하고 기준선을 유지한다. 성능 수치를 꾸미는 것보다 조건과 한계를 설명하는 보고서가 중요하다.

09 / 6주 진행

한 주에 산출물 하나씩 완성한다

주 6~8시간을 이 CPP 폴더에 집중할 때의 제안 일정이다.

세 폴더를 동시에 공부한다면 아래 시간을 합산하지 말고 전체 가용 시간 안에서 기간을 늘린다. 한 주는 읽기 1~2시간, 구현 3~4시간, 테스트·회고 2시간으로 시작한다. 장비 연결이 막혀도 core와 fake 작업은 계속 진행할 수 있다. 다음 단계는 날짜가 아니라 통과 기준으로 결정한다.

주	완성할 것	건너뛰기 또는 통과 기준
1	CMake/CTest + ledger 최소 프로젝트	다른 경로에서 빌드 성공·의도한 테스트 실패를 재현하면 빌드 입문 학습 생략
2	Frame 검증과 오류·소유권 계약	잘못된 크기·overflow·거부 뒤 상태 불변 테스트가 이미 있으면 adapter 확장
3	stop 완료와 Faulted·새 세션 규칙	callback 보류·중복 stop·늦은 결과 테스트가 있으면 복구 조건만 보완
4	정산식과 결정적 종료 경쟁 테스트	full/empty/running 중 취소 결과가 일관되면 stress 조합으로 확장
5	장애 재현 보고서와 성능 기준선	재현 명령·원시 trace·CSV가 있으면 병목 한 개만 개선
6	독립 실행 패키지와 면접 설명	다른 사람이 README만으로 재현하고 설계 판단을 질문할 수 있으면 마무리

주마다 남길 짧은 기록

- 새로 확인한 규칙 1개: 예를 들어 stop 요청과 Stopped는 다른 사건이다.
- 깨뜨려 본 가정 1개: 큐 용량만 제한해도 전체 메모리가 제한된다는 가정을 반례로 확인한다.
- 실행 가능한 증거 1개: 테스트 이름, 입력, 실행 명령, 로그 위치를 적는다.

C++ 문법이 막히면 필요한 범위만 00·03·06으로 돌아간다. 알고리즘의 정확도, 캘리브레이션, 결함 검출 평가는 CV 자료에서 다루고, 여기서는 그 알고리즘을 안정적으로 호출·취소·관측하는 시스템 경계를 끝낸다. 외부 SDK가 생기면 fake에서 세운 계약을 제조사 문서와 실제 동작에 대조한다.

일정이 밀릴 때

UI·실카메라·고급 최적화를 뒤로 미루고 콘솔 + fake + 정산·종료 테스트를 먼저 완성한다. 이미 통과한 기본 주제의 요약 노트를 늘리는 데 시간을 쓰지 않는다.

10 / 이직에 쓸 증거

설계 선택을 코드와 결과로 설명한다

학습량보다 자신이 맡은 문제, 실패 조건, 검증 방법이 보이게 정리한다.

산출물	리뷰어가 확인할 내용
README + build 설정	필수 도구 버전, 새 경로 실행 명령, SDK 없는 실행 방법
계약·상태 전이 문서	Frame 유효 조건, 소유권, stop 완료, Faulted와 재접속 조건
테스트와 장애 보고서	수락한 작업의 유실·중복 방지, 의도적으로 실패시키는 방법
성능 비교 보고서	같은 입력·설정의 기준선과 변경본, 원시 자료, 분리한 결과도 포함

면접 질문에 답하는 방식

- “왜 shared_ptr로 전부 해결하지 않았나요?” - 객체 수명 연장과 SDK callback 중지 보장은 다르다. 대여·반납 계약과 종료 테스트로 설명한다.
- “프레임이 유실되지 않았다는 근거는?” - 앱 관측 범위를 먼저 밝히고 received/accepted/terminal 정산식과 카메라별 로그를 보여 준다.
- “종료가 늦으면 어떻게 하나요?” - 취소 가능 구간과 외부 호출을 구분하고 deadline 초과 시 Faulted와 남은 자원 처리 정책을 설명한다.
- “얼마나 빨라졌나요?” - 실제 측정한 입력·환경·거부율·지연 분포를 함께 제시한다. 아직 측정하지 않았으면 숫자를 쓰지 않는다.

이력서 문장은 실제 실행 뒤에 작성한다. 예: ‘카메라 입력 대역과 C++ 검사 core를 분리하고, 큐 포화·종료 경쟁을 재현하는 테스트와 작업 정산 지표를 구현했다.’ 성능 개선률이나 무장애 운영 기간은 검증 자료가 있는 경우에만 덧붙인다. 회사 코드·현장 이미지 대신 직접 만든 입력과 공개 가능한 구현을 사용한다.

최종 완료 조건

재현 빌드, 잘못된 입력 거부, 작업 정산, 종료 경쟁, 늦은 세션 결과 차단, 실패 보고서가 모두 있어야 한다. 개념을 설명할 수 있어도 아직 코드와 기록이 없다면 해당 단계는 진행 중이다.

검증 범위: 이 문서는 기존 노트·PDF 내용과 공식 문서를 바탕으로 정적 검토한 학습용 설계다. 새 C++ 예제를 이 환경에서 컴파일·실행한 결과를 제공하는 것은 아니다. 코드 골격은 구현 실습의 출발점이며 실제 SDK의 callback·buffer·취소 계약은 별도로 확인해야 한다. 시간·횟수 기준은 채용 기준이 아닌 제안이다.

APPENDIX / LOCAL NOTES

어떤 자료에서 이어졌나

경로는 표지의 기준 폴더 안에서 읽습니다. 기존 노트를 함께 펴놓고 이어서 학습하세요.

00.메모리·객체 수명·RAII.md

소유권, 이동, 예외 안전성, sanitizer, 메모리 지역성까지 설명한다. RAII 정의를 다시 공부하기보다 경계 입력 검증과 실패 후 상태 보존으로 확장한다.

01.프로세스와 스레드.md

실행 모델, 메모리 모델, 동기화, 협력적 취소를 설명한다. 이 지식을 서비스 상태 전이와 종료 증명에 적용한다.

02.동시성과 병렬성.md

backpressure, Amdahl 법칙, 성능 기준선, 동시성 테스트를 다룬다. 추가본에서는 실제 카운터 정산과 같은 입력의 비교 실험을 만든다.

03.Mutex와 Condition Variable.md

종료 가능한 bounded queue와 불변식, deadline, stress test가 이미 있다. 큐 재구현보다 프레임 전체 생애의 정산과 결정적 실행 순서를 추가한다.

04.C++ 메모리와 객체 수명 보완.md

할당기·단편화·주소 변환과 소유권 전이를 보완한다. 전체 프레임 메모리 예산 및 측정 근거를 만드는 단계로 이어간다.

05.파일·소켓·비동기 IO와 Buffer.md

SDK 버퍼의 세 가지 계약과 종료 상태 머신이 있다. SDK 완료 보장, 타임아웃 뒤 상태, 재접속 세션 격리를 구체화한다.

검토 기준

기존 자료의 주제와 실습을 검토해 다음 과제를 골랐습니다. 문서에서 확인되지 않는 내용은 자료상의 빈칸으로 해석하며, 개인의 실제 경험이나 숙련도를 단정하지 않습니다.

APPENDIX / LOCAL NOTES

어떤 자료에서 이어졌나 · 계속

경로는 표지의 기준 폴더 안에서 읽습니다. 기존 노트를 함께 펴놓고 이어서 학습하세요.

06.C++ 객체지향 설계와 테스트 가능한 시스템.md

의존성 분리, fake clock, InlineExecutor, 오류 표현과 thread component test가 있다. 외부 장비 없이 동일 실패를 다시 실행하는 테스트 패키지로 확장한다.

07.비동기 작업 취소와 UI 이미지 처리.md

latest-wins, generation, UI commit 검사와 worker 예외 경계를 이미 다룬다. 작업 완료와 UI 표시를 서로 다른 관측값으로 정산한다.

08.다중 카메라 이미지 검사 스레드 설계와 멈춤 진단.md

통합 예제와 진단 절차가 있다. 7.2절에 남은 callback barrier, drop 계수, stride/format 검증을 이번 추가 실습의 출발점으로 삼는다.

PDF/01_CPP_메모리_객체수명_설계.pdf

67쪽. 00:04:06 노트를 묶은 본문을 텍스트 추출해 기존 범위를 확인했다.

PDF/02_프로세스_스레드_동시성_동기화.pdf

72쪽. 01:02:03 노트에 해당하는 실행·동기화 내용을 확인했다.

PDF/03_IO_비동기_UI_다중카메라.pdf

73쪽. 05:07:08 노트에 해당하는 비동기·카메라 내용을 확인했다.

REFERENCES / PRIMARY SOURCES

참고 자료와 활용 범위

본문의 [번호]에 대응하는 공식 문서·원문입니다. 확인일: 2026.09.26.

[1] CMake - Buildsystem

target별 build 조건과 PUBLIC/PRIVATE/INTERFACE 전파 규칙. 학습단원 02의 빌드 설계 근거.

<https://cmake.org/cmake/help/latest/manual/cmake-buildsystem.7.html>

[2] CMake - Presets

공통 configure/build/test preset과 개인 설정 파일 구분. 학습단원 02 적용.

<https://cmake.org/cmake/help/latest/manual/cmake-presets.7.html>

[3] Microsoft - CancellationEx

취소 요청과 완료의 구분, 완료 전 자원 수명 유지. 학습단원 04 적용.

<https://learn.microsoft.com/en-us/windows/win32/api/ioapiset/nf-ioapiset-cancelioex>

[4] Microsoft - AddressSanitizer

MSVC의 메모리 오류 검사와 지원 범위. 학습단원 07 적용.

<https://learn.microsoft.com/en-us/cpp/sanitizers/asan?view=msvc-170>

[5] LLVM/Clang - ThreadSanitizer

data race 탐지와 지원 OS 확인. 학습단원 07의 별도 Linux/Clang core 검사 선택 근거.

<https://clang.llvm.org/docs/ThreadSanitizer.html>